

HW 3

4.2.2

f

$S \rightarrow a S b S \mid b S a S \mid \epsilon$

with string *aabbab*.

a

Give a leftmost derivation for the string

✓ Answer ✓

1. $a S b S$

l. $a (a S b S) b S$

A. $a (a (b S a S) b S) b S$

i. $a (a (b (b S a S) a S) b S) b S$

a. $a (a (b (b (a S b S) a S) a S) b S) b S$

01. $a (a (b (b (a (b S a S) b S) a S) a S) b S) b S$

• Failed

02. $a (a (b (b (a \in b S) a S) a S) b S) b S$

• Failed

• Failed

ii. $a (a (b (b \in a S) a S) b S) b S$

a. $a (a (b (b \in a (b S a S)) a S) b S) b S$

01. $a (a (b (b \in a (b \in a S)) a S) b S) b S$

• Failed

• Failed

b. $a (a (b (b \in a \epsilon) a S) b S) b S$

• Failed

• Failed

• Failed

B. $a (a \in b S) b S$

i. $a (a \in b (b S a S)) b S$

a. $a (a \in b (b (a S b S) a S)) b S$

01. $a (a \in b (b (a (b S a S) b S) a S)) b S$

01. $a (a \in b (b (a (b \in a S) b S) a S)) b S$

• Failed

- Failed

02. $a (a \in b (b (a \in b S) a S)) b S$

01. $a (a \in b (b (a \in b \epsilon) a S)) b S$

- Failed

- Failed

- Failed

b. $a (a \in b (b \in a S)) b S$

01. $a (a \in b (b \in a (b S a S))) b S$

01. $a (a \in b (b \in a (b \in a S))) b S$

- Failed

- Failed

02. $a (a \in b (b \in a \epsilon)) b S$

01. $a (a \in b (b \in a \epsilon)) b \epsilon$

- Succeeded

b

Give a rightmost derivation for the string

✓ **Answer**

1. $a S b \epsilon$

I. $a (b S a \epsilon) b \epsilon$

- Failed

II. $a (a S b S) b \epsilon$

A. $a (a S b (b S a \epsilon)) b \epsilon$

i. $a (a \in b (b \in a \epsilon)) b \epsilon$

- Succeeded

c

Give a parse tree for the string

✓ **Answer**

```

S \- a
  \- S \- a
    \- S \- e
      \- b
        \- S \- b
          \- S \- e

```

```

|           ⊢ a
|           ⊢ S \- e
⊢ b
⊢ S \- e

```

4.3.1

The following is a grammar for regular expressions over symbols a and b only, using $+$ in place of $|$ for union, to avoid conflict with the use of vertical bar as a metasymbol in grammars:

$$\begin{aligned}
 rexr &\rightarrow rexr + rterm \mid rterm \\
 rterm &\rightarrow rterm rfactor \mid rfactor \\
 rfactor &\rightarrow rfactor * \mid rprimary \\
 rprimary &\rightarrow a \mid b
 \end{aligned}$$

a

Left factor this grammar

✓ **Answer**

Already left factored

b

Does left factoring make the grammar suitable for top-down parsing?

✓ **Answer**

No since it is still left-recursive.

c

In addition to left factoring, eliminate left recursion from the original grammar.

✓ **Answer**

$$\begin{aligned}
rexpr &\rightarrow rterm\ rexpr' \\
rexpr' &\rightarrow +rterm\ rexpr' \mid \epsilon \\
rterm &\rightarrow rfactor\ rterm' \\
rterm' &\rightarrow rfactor\ rterm' \mid \epsilon \\
rfactor &\rightarrow rprimary\ rfactor' \\
rfactor' &\rightarrow *rfactor' \mid \epsilon \\
rprimary &\rightarrow a \mid b
\end{aligned}$$

d

Is the resulting grammar suitable for top-down parsing?

✓ **Answer**

Yes

4.4.4

Compute **FIRST** and **FOLLOW** for each of the grammars

f

$S \rightarrow a S b S \mid b S a S \mid \epsilon$

with string *aabbab*.

✓ **Answer**

FIRST = $\{a, b, \epsilon\}$

FOLLOW = $\{a, b, \$\}$

Compiler

Notes on the error handling and refactoring are in the submitted code at `assignment/week3.md`.